EE5141: Introduction to Wireless and Cellular Communication

Simulation Assignment 2 - Report

January - May 2026

Name: Vignesh Natarajkumar
Roll No.: EE23B087

Introduction

The simulations follow the system model: an OFDM system with $N = 512$, subcarrier spacing 10 kHz, sampling rate 5.12 Msps, cyclic prefix duration 6.25 μs, and a frame size of five OFDM symbols. The first OFDM block in each frame is the synchronization preamble and the next four carry pilot-assisted QPSK data.

The common multipath model used in Questions 1 to 3 is the six-tap channel with delay samples $[0, 7, 13, 18, 21, 26]$ and average powers in dB $[-3, 0, -1, -4, -9, -17]$. These powers are first converted to linear scale and then normalized so that the total average channel gain is unity. Since the maximum excess delay is 26 samples and the cyclic prefix length is $N_{CP} = 32$, the system can operate without IBI or ICI once timing and carrier-frequency synchronization are correct.

Parameter	**Value**
FFT size	$N = 512$
Subcarrier spacing	$f_{sub} = 10$ kHz
Useful OFDM symbol duration	$T = 1/f_{sub} = 100$ μs
Sampling rate	$F_s = 5.12$ Msps
Sampling period	$T_s = 1/F_s = 195.3125$ ns
Cyclic prefix duration	$T_{CP} = 6.25$ μs $= 32$ samples
OFDM symbol duration	$T_{OFDM} = 106.25$ μs
Useful subcarriers	$n \in \{-240, \ldots, -1, 1, \ldots, 240\}$
Pilot subcarriers in Question 3	$n \in$ $\{-240, -232, \ldots, -8, 8, \ldots, 232, 240\}$
Number of pilot subcarriers	$N_P = 60$
Number of useful subcarriers	$N_U = 480$

Table 1: System constants used in all simulations.

Question 1

Question: SC Frequency Sync for a frequency-selective channel: design a preamble that allows the Schmidl-Cox algorithm to estimate the full carrier-frequency offset range $\Delta f = \pm 28.65$ kHz. Then plot the preamble properties and the CFO-estimation MSE for $J = 10$ trials as SNR varies from 0 dB to 20 dB in 2 dB steps.

Answer:
The Schmidl-Cox (SC) CFO estimator uses the phase change between two repeated blocks in the received preamble to obtain the carrier frequency offset. If the repetition length is L samples, then the SC correlation has phase

$$\angle P = 2\pi \Delta f L T_s,$$

and the corresponding CFO estimate is

$$\hat{\Delta f} = \frac{\angle P}{2\pi L T_s}.$$

This estimate is unambiguous only if

$$|\Delta f| < \frac{1}{2LT_s}.$$

Any frequency offset that causes an accumulated phase shift beyond $\pm\pi$ will wrap around.
If the standard half-symbol repetition were used, then $L = N/2 = 256$ and the unambiguous CFO range would become only

$$\frac{1}{2 \cdot 256 \cdot T_s} = 10 \text{ kHz},$$

which is insufficient for the required ± 28.65 kHz offset. Therefore the preamble must contain a shorter repetition period.

(a) Preamble design and unambiguous CFO range

The implemented preamble activates only every eighth useful subcarrier and places i.i.d. QPSK symbols on those active tones. Since the occupied carriers form an 8-spaced comb in the frequency domain, the time-domain OFDM symbol is periodic with period

$$L = \frac{N}{8} = 64 \text{ samples}.$$

This automatically implies a two-half repetition as well, because a 64-sample periodic sequence is also periodic over 256 samples.
The maximum unambiguous CFO range therefore becomes

$$|\Delta f| < \frac{1}{2 \cdot 64 \cdot T_s} = 40 \text{ kHz},$$

which safely covers ± 28.65 kHz. There are exactly 60 active preamble tones: 30 on the negative-frequency side and 30 on the positive-frequency side. The DC carrier remains unused.
The plot of the frequency-domain and time-domain preambles are given below:

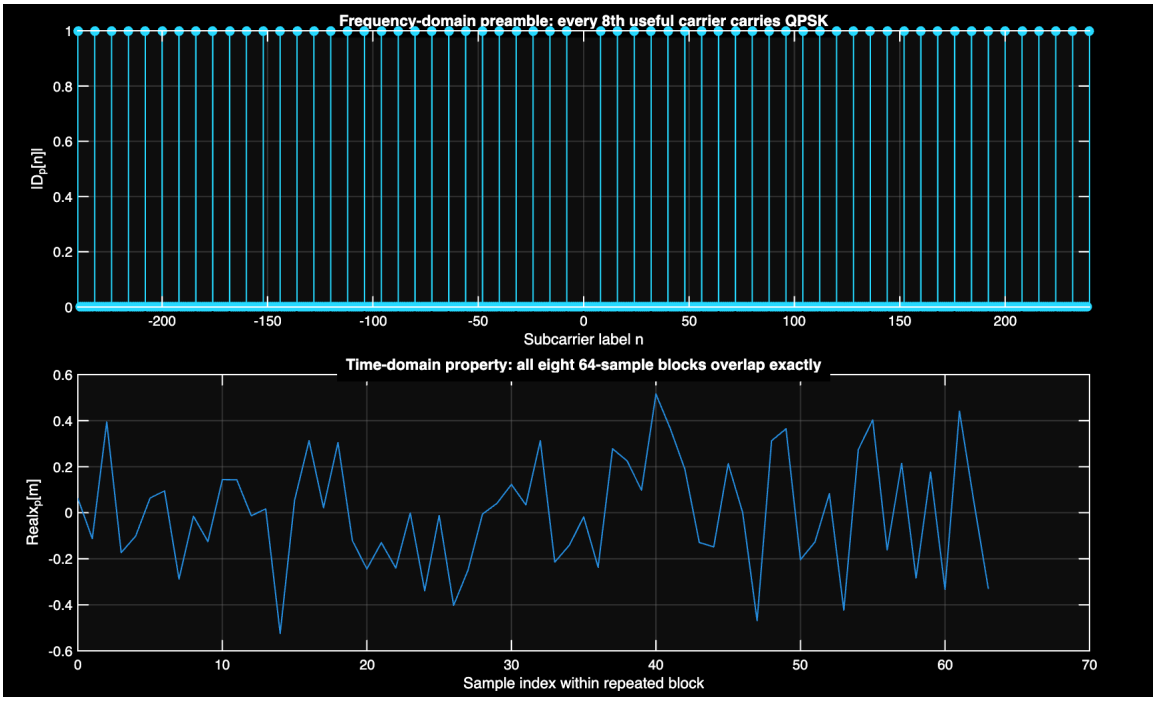

Figure 1: Question 1(a): Frequency-domain comb preamble and the corresponding 64-sample periodic time-domain structure.

(b) CFO-estimation MSE versus SNR

For the MSE experiment, the script performs $J = 10$ independent trials at each SNR point from 0 dB to 20 dB in 2 dB steps. In each trial:

1. a new channel realization is drawn from the given PDP,
2. a CFO is drawn uniformly from the interval $[-28.65, 28.65]$ kHz,
3. noise is added to achieve the target SNR,
4. the SC CFO estimator with $L = 64$ is applied,
5. the squared error $(\Delta f - \hat{\Delta f})^2$ is accumulated.

The plotted curve is below:

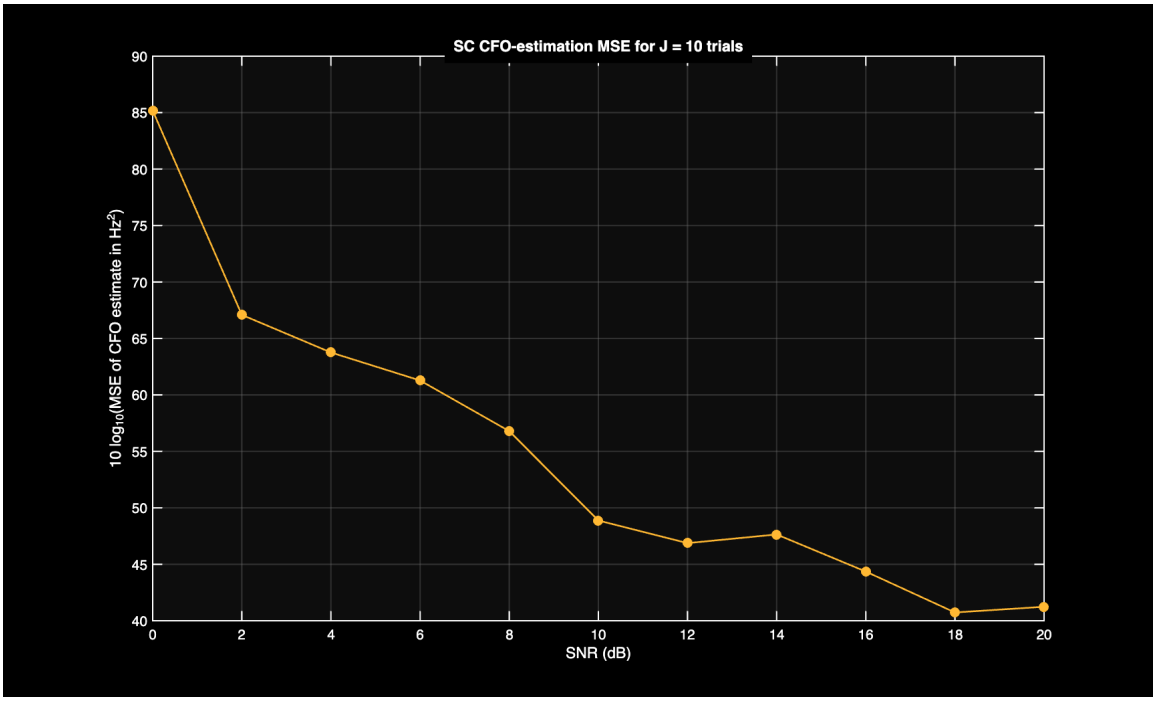

Figure 2: Question 1(b): CFO-estimation MSE curve for SNR values from 0 dB to 20 dB.

The main interpretations are:

- At low SNR, the SC correlation phase is noisy and the CFO estimate has a large error variance.
- As SNR increases, the MSE drops because the repeated structure becomes easier to detect reliably. More Monte Carlo simulations will help to smoothen the curve.
- The estimator remains valid across the full required offset interval because the preamble was designed around the unambiguous CFO constraint. We have also seen how to design frequency-domain preambles to ensure that a CFO bound can be achieved through this.

Question 2

Question: Using the same preamble from Question 1, obtain the SC timing metric at SNR = 6 dB over two consecutive OFDM frames: first for a single-tap channel and then for the given multipath channel. Plot the metric from which the FFT window is derived and comment on the result.

Answer:
Because the preamble designed in Question 1 is periodic over 64 samples, it inherently contains two identical 256-sample halves ($N/2$). Therefore, the standard Schmidl-Cox timing correlation can be utilized to maximize the captured signal energy, defining the lag distance as $N/2 = 256$ samples.
The receiver operates continuously on the incoming sample stream $r[d]$, maintaining a sliding window to compute the timing metric $M(d)$:

$$M(d) = \frac{|P(d)|^2}{(R(d))^2 + \varepsilon}$$

where $P(d)$ is the cross-correlation between the two halves of the sliding window, and $R(d)$ normalizes the metric by the received energy in the second half:

$$P(d) = \sum_{m=0}^{N/2-1} r^*[d+m]r[d+m+N/2] \quad \text{and} \quad R(d) = \sum_{m=0}^{N/2-1} |r[d+m+N/2]|^2$$

The idea is to use the periodicity in the time-domain due to the preamble structure to wait for an exact repetition at which point the metric will shoot up.
However, because the preamble also carries a cyclic prefix, the raw timing metric does not ideally produce a single sharp spike. Instead it forms a plateau whose width is related to the valid CP-supported timing region.
To obtain a practical FFT boundary, I smooth the raw metric over N_{CP} samples and then shift the center of the high-metric region by approximately $N_{CP}/2$ samples to place the derived FFT window near the useful-symbol boundary.

(a) Single-tap channel

In the single-tap case, since there is no delay spread, the repeated halves align very well and the correlation plateau is narrow and well defined. The CP-smoothed decision metric provides a clear location from which the FFT window can be derived.

(b) Multipath channel

Here, the timing metric becomes broader and less sharply peaked. This is a direct consequence of the delayed signal replicas: several shifted versions of the preamble contribute energy to the correlation, so the transition into and out of the plateau is less abrupt.
But it is important to note that multipath does not make SC timing fail here, because the CP is still longer than the maximum excess delay, so ISI is prevented. However, multipath does make the metric more ambiguous if one tries to locate the timing by using the raw maximum alone.
The timing results for both the channel variants is given below:

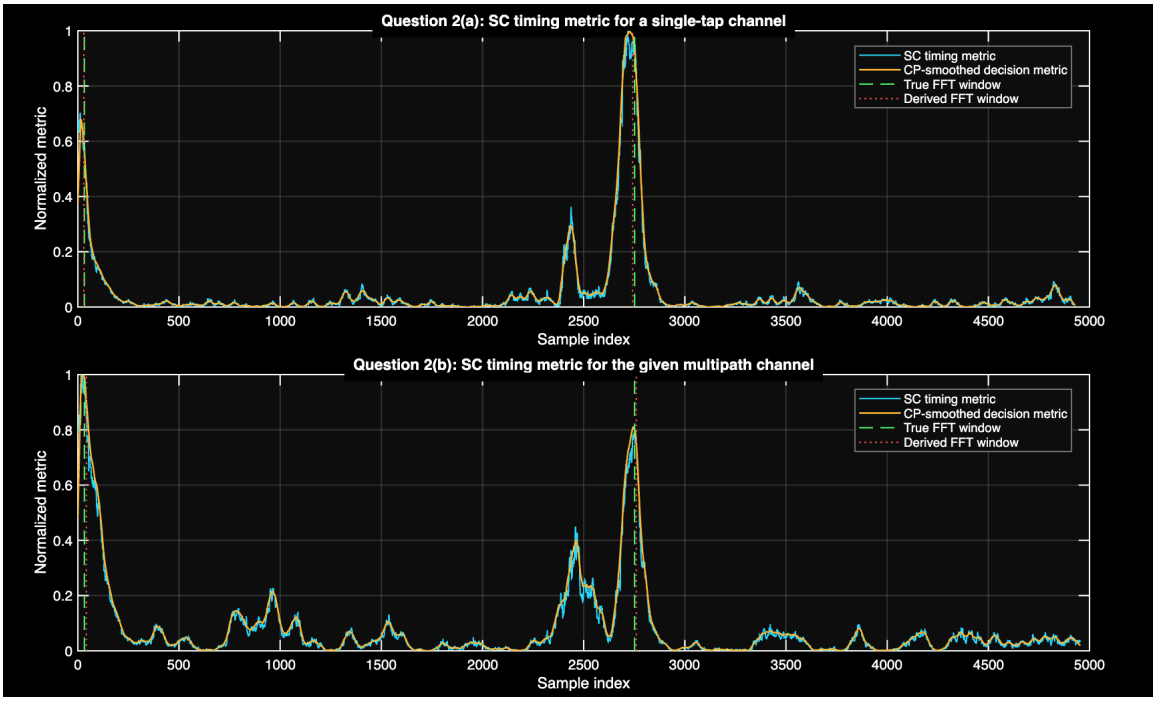

Figure 3: Question 2: SC timing metric and CP-smoothed decision metric over two consecutive frames for the single-tap and multipath cases.

Some conclusions are:

- In the single-tap case the metric plateau is narrower and more regular.
- In the multipath case the plateau is wider, but the estimation of the FFT window is still good.
- In general the SC timing detection process is a plateau-based decision rather than a single-sample peak detector.

Question 3

Question: For the pilot-assisted OFDM data symbols following the preamble, estimate the channel using: (a) ZF on pilot tones, (b) linear interpolation to all useful tones, (c) FFT-based interpolation and one bonus enhancement, (d) modified LS with CIR support limited by the CP, and (e) an improved estimator when the exact path delays are known. Plot all MSE curves on the same graph for SNR values from 0 dB to 30 dB in 3 dB steps using $J = 100$ independent channel realizations.

Answer:
The pilot placement for this OFDM resource is done as per the last 2 questions: every eighth useful subcarrier starting from $n = 240$, with the DC subcarrier always excluded. This gives a pilot vector on the non-zero subcarriers

$$n \in \{-240, -232, \ldots, -8, 8, \ldots, 232, 240\},$$

which corresponds to $N_P = 60$ pilot tones inside the total useful band of $N_U = 480$ subcarriers.
The FFT-domain pilot measurement model is

$$Y[k, n] = X[k, n]G[k, n] + V[k, n].$$

On pilot locations, the receiver therefore observes a noisy version of the channel frequency response (CFR). The estimators differ only in how they extend this noisy information to the full useful band.

(a) Zero-forcing estimate on pilot locations

The most direct estimate is the pilot-by-pilot zero-forcing estimate

$$\hat{G}_{ZF}(n_p) = \frac{Y(n_p)}{X(n_p)}.$$

This removes the known QPSK pilot symbols exactly but does not reduce noise in any way. As a result, this estimator is unbiased on the pilot tones but noisy. It will naturally not work well for low SNR. However, note that since the MSE for this method alone is being computed for the pilot locations, avoiding any interpolation error, the plot below will still show agreeable performance.

(b) Linear interpolation over all useful tones

To obtain the Channel Frequency Response (CFR) across the data subcarriers, the simplest approach is to linearly interpolate the real and imaginary parts of $\hat{H}_{ZF}$ across all useful tones. This exploits the basic property that a multipath channel with a limited delay spread exhibits a CFR that changes smoothly across frequency. However, the channel is not perfectly linear between adjacent pilots, so interpolation error remains even when noise becomes small.
This is why the linearly interpolated curve improves over raw pilot-only ZF at low and moderate SNR but usually exhibits an error floor at high SNR. Once the additive noise becomes small, the remaining error is dominated by interpolation error, not by noise.

(c) FFT-based interpolation and bonus

Here, the N_P zero-forcing pilot estimates are mapped to their exact subcarrier bins in an N-point grid, with all empty data bins set to zero. Taking the N-point IFFT of this sparse comb yields a time-domain impulse response containing repeated aliases of the true channel.Because the physical

channel length is strictly bounded by the cyclic prefix, this impulse response is truncated to the first $N_{CP} = 32$ taps, and a scaling factor (equal to the pilot spacing, 8) is applied to restore the true signal energy. Padding these 32 taps with zeros and applying an N-point FFT reconstructs the full-band CFR.
For the bonus enhancement in the simulation, a raised-cosine tapering window is applied to the final 8 taps of the retained support before taking the FFT. This softens the hard truncation edge and mildly reduces spectral leakage.

(d) Modified least squares (mLS)

The modified least-squares estimator assumes only that the channel impulse response is contained within the first 32 taps. Let $\mathbf{z}_{ZF}$ denote the pilot-domain ZF vector. Then the pilot-domain model can be written as

$$\mathbf{z}_{ZF} \approx \mathbf{A}_P \mathbf{h},$$

where $\mathbf{h} \in \mathbb{C}^{32}$ is the unknown delay-domain channel vector and $\mathbf{A}_P$ is the partial DFT matrix that maps these 32 taps to the 60 pilot positions. The mLS estimate is

$$\hat{\mathbf{h}}_{mLS} = \arg\min_{\mathbf{h}} \|\mathbf{z}_{ZF} - \mathbf{A}_P \mathbf{h}\|_2^2,$$

followed by CFR reconstruction on all useful tones.
This method is fundamentally better conditioned than interpolation because it solves for the channel in the domain where the number of unknowns is only 32, not 480.

(e) Delay-aware least squares with known tap locations

If the receiver knows the exact path delays $[0, 7, 13, 18, 21, 26]$, then the parameter dimension can be reduced even further. Instead of estimating all 32 possible CP-limited taps, the receiver needs to estimate only the 6 truly active tap gains. The pilot-domain model becomes

$$\mathbf{z}_{ZF} \approx \mathbf{A}_{P,known} \mathbf{b},$$

where $\mathbf{b} \in \mathbb{C}^6$ contains only the complex gains at the known delay positions. This is the strongest estimator because it uses the most accurate prior information.
The plot representing the performance of these methods is given below:

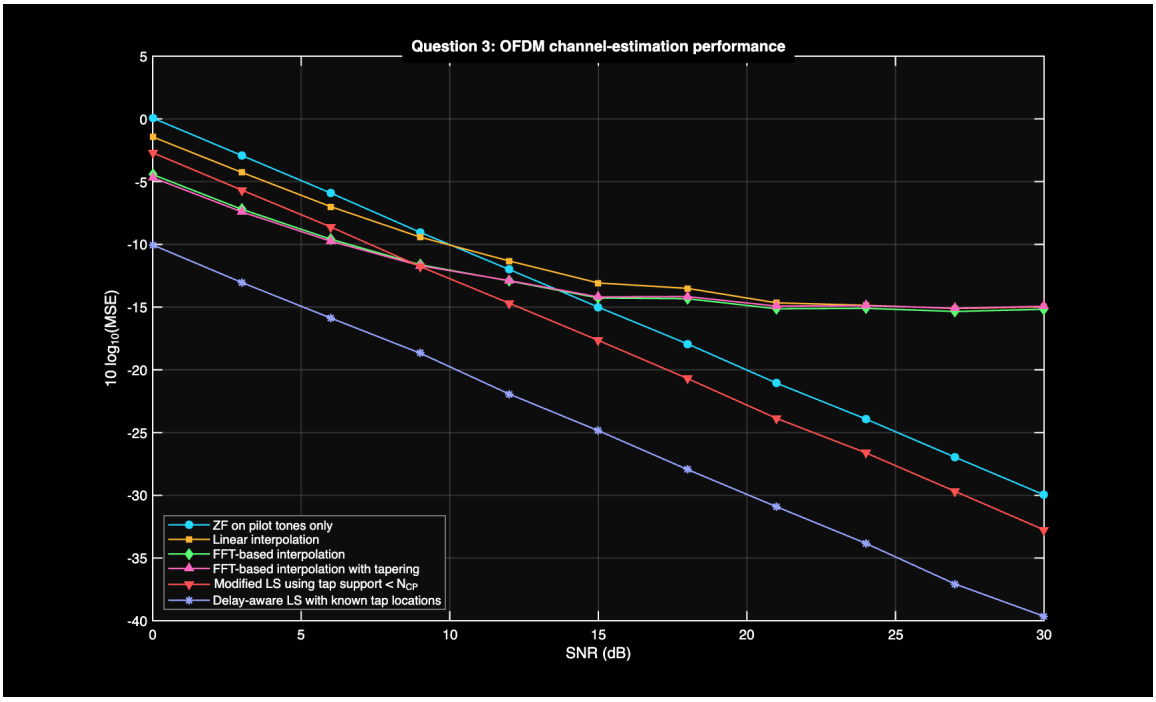

Figure 4: Question 3: MSE curves for ZF on pilots, linear interpolation, FFT-based interpolation, tapered FFT interpolation, modified LS, and delay-aware LS.

One additional behavior is important at high SNR: the purely interpolation-based methods can flatten out into an error floor, whereas the structurally informed LS methods continue improving. This is a direct consequence of the fact that interpolation error is not noise; it remains even when the pilot noise becomes negligible.
The reconstructed CFR based on the estimates obtained by the methods above is shown in the below plot:

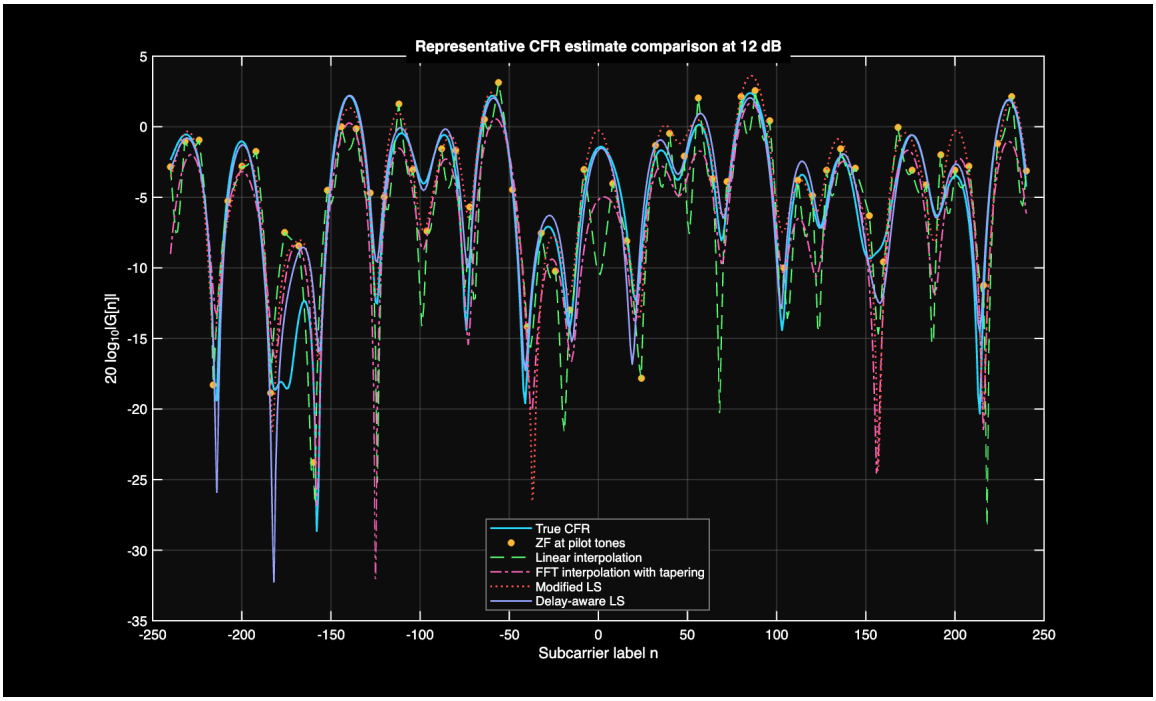

Figure 5: Representative CFR realization at 12 dB showing the true CFR, noisy ZF pilot samples, and the reconstructed CFR for the main estimation methods.

The representative CFR plot is useful for interpretation. It shows that:

- the ZF pilot samples are visibly noisy,
- linear interpolation can miss sharper spectral curvature between pilots,
- the delay-aware and mLS estimates follow the true CFR more closely across the full useful band,
- the FFT-based methods reduce noise well.

A MATLAB Codes

Question1.m

```
clear;
clc;
close all;

rng(5141, "twister");
params = getSystemParameters();
theme = getPlotTheme();
[preambleFd, preambleTd, preambleWithCp, activeSubcarriers] = buildRepeatedPreamble(params);

repetitionLag = params.N / 8;
maxUnambiguousCfoHz = 1 / (2 * repetitionLag * params.Ts);

figure("Name", "Question 1(a)", "Color", theme.figureColor);
tiledlayout(2, 1, "TileSpacing", "compact", "Padding", "compact");

ax1 = nexttile;
ax1.ColorOrder = theme.palette;
freqProfile = zeros(size(params.usefulSubcarriers));
activeMask = mod(params.usefulSubcarriers, 8) == 0;
freqProfile(activeMask) = abs(preambleFd(params.usefulCarrierBins(activeMask)));
stem(params.usefulSubcarriers, freqProfile, "filled", "LineWidth", 1.0, ...
    "Color", theme.palette(1, :), "MarkerFaceColor", theme.palette(1, :), ...
    "MarkerEdgeColor", theme.palette(1, :));
grid on;
xlim([-240 240]);
xlabelHandle = xlabel("Subcarrier label n");
ylabelHandle = ylabel("|D_p[n]|");
titleHandle = title("Frequency-domain preamble: every 8th useful carrier carries QPSK");
applyDarkAxes(ax1, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);

ax2 = nexttile;
ax2.ColorOrder = theme.palette;
repeatedBlocks = reshape(preambleTd, repetitionLag, []);
plot(0:repetitionLag-1, real(repeatedBlocks), "LineWidth", 1.0);
grid on;
xlabelHandle = xlabel("Sample index within repeated block");
ylabelHandle = ylabel("Real{x_p[m]}");
titleHandle = title("Time-domain property: all eight 64-sample blocks overlap exactly");
applyDarkAxes(ax2, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);

fprintf("Question 1(a)\n");
fprintf("Active preamble subcarriers = %d\n", numel(activeSubcarriers));
fprintf("Repeated block length = %d samples\n", repetitionLag);
fprintf("Maximum unambiguous CFO = +/- %.2f kHz\n\n", maxUnambiguousCfoHz / 1e3);

snrDbList = 0:2:20;
numTrials = 10;
mseVsSnr = zeros(size(snrDbList));

for snrIdx = 1:numel(snrDbList)
    snrDb = snrDbList(snrIdx);
    squaredErrors = zeros(numTrials, 1);

    for trialIdx = 1:numTrials
        channelImpulseResponse = drawChannel(params);
        cfoTrueHz = -params.maxOffsetHz + 2 * params.maxOffsetHz * rand;
```

```
        receivedClean = conv(preambleWithCp, channelImpulseResponse);
        noiseVariance = 10^(-snrDb / 10);
        noise = sqrt(noiseVariance / 2) * ...
            (randn(size(receivedClean)) + 1j * randn(size(receivedClean)));
        sampleIndex = (0:numel(receivedClean)-1).';
        received = exp(1j * 2 * pi * cfoTrueHz * sampleIndex * params.Ts) .* ...
            (receivedClean + noise);

        receivedWithoutCp = received(params.cpLen+1:params.cpLen+params.N);
        cfoEstimateHz = estimateScCfo(receivedWithoutCp, params, repetitionLag);
        squaredErrors(trialIdx) = abs(cfoTrueHz - cfoEstimateHz)^2;
    end

    mseVsSnr(snrIdx) = mean(squaredErrors);
end

figure("Name", "Question 1(b)", "Color", theme.figureColor);
ax3 = axes();
plot(ax3, snrDbList, 10 * log10(mseVsSnr + eps), "o-", ...
    "LineWidth", 1.2, "MarkerSize", 6, "Color", theme.palette(2, :), ...
    "MarkerFaceColor", theme.palette(2, :), "MarkerEdgeColor", theme.palette(2, :));
grid on;
xlabelHandle = xlabel("SNR (dB)");
ylabelHandle = ylabel("10 log_{10}(MSE of CFO estimate in Hz^2)");
titleHandle = title("SC CFO-estimation MSE for J = 10 trials");
applyDarkAxes(ax3, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);

fprintf("Question 1(b)\n");
fprintf("The script draws CFO uniformly from [-28.65, 28.65] kHz in every trial.\n");
fprintf("The plotted MSE is in Hz^2 before conversion to dB.\n");

function params = getSystemParameters()
params.N = 512;
params.subcarrierSpacing = 10e3;
params.bandwidth = 5.12e6;
params.Ts = 1 / params.bandwidth;
params.cpLen = round(6.25e-6 / params.Ts);
params.frameSymbols = 5;
params.symbolLen = params.N + params.cpLen;
params.maxOffsetHz = 28.65e3;
params.usefulSubcarriers = [-240:-1, 1:240];
params.usefulCarrierBins = mod(params.usefulSubcarriers, params.N) + 1;
params.pathDelays = [0 7 13 18 21 26];
pathPowersDb = [-3 0 -1 -4 -9 -17];
pathPowersLinear = 10.^(pathPowersDb / 10);
params.pathPowers = pathPowersLinear / sum(pathPowersLinear);
end

function [freqGrid, timeBlock, blockWithCp, activeSubcarriers] = buildRepeatedPreamble(params)
activeSubcarriers = params.usefulSubcarriers(mod(params.usefulSubcarriers, 8) == 0);
freqGrid = zeros(params.N, 1);
freqGrid(mod(activeSubcarriers, params.N) + 1) = randomQpsk(numel(activeSubcarriers));
timeBlock = ifft(freqGrid) * sqrt(params.N);
blockWithCp = [timeBlock(end-params.cpLen+1:end); timeBlock];
end

function qpskSymbols = randomQpsk(symbolCount)
qpskSymbols = exp(1j * (pi / 4 + (pi / 2) * randi([0 3], symbolCount, 1)));
end

function channelImpulseResponse = drawChannel(params)
channelImpulseResponse = zeros(params.pathDelays(end) + 1, 1);
taps = sqrt(params.pathPowers(:) / 2) .* ...
    (randn(numel(params.pathDelays), 1) + 1j * randn(numel(params.pathDelays), 1));
channelImpulseResponse(params.pathDelays + 1) = taps;
```

```
end

function cfoEstimateHz = estimateScCfo(receivedBlock, params, lagSamples)
correlationValue = sum(conj(receivedBlock(1:end-lagSamples)) .* receivedBlock(1+lagSamples:end));
cfoEstimateHz = angle(correlationValue) / (2 * pi * lagSamples * params.Ts);
end

function theme = getPlotTheme()
theme.figureColor = [0.00 0.00 0.00];
theme.axesColor = [0.05 0.05 0.05];
theme.textColor = [1.00 1.00 1.00];
theme.gridColor = [0.45 0.45 0.45];
theme.palette = [ ...
    0.15 0.85 1.00; ...
    1.00 0.72 0.20; ...
    0.35 1.00 0.45; ...
    1.00 0.40 0.75; ...
    1.00 0.32 0.32; ...
    0.60 0.65 1.00; ...
    0.20 1.00 0.85; ...
    1.00 0.92 0.30];
end

function applyDarkAxes(ax, theme)
set(ax, "Color", theme.axesColor, ...
    "XColor", theme.textColor, ...
    "YColor", theme.textColor, ...
    "GridColor", theme.gridColor, ...
    "MinorGridColor", theme.gridColor, ...
    "GridAlpha", 0.35, ...
    "MinorGridAlpha", 0.18, ...
    "LineWidth", 1.0);
box(ax, "on");
end

function styleTitle(titleHandle, theme)
set(titleHandle, "Color", theme.textColor, ...
    "BackgroundColor", [0.00 0.00 0.00], ...
    "EdgeColor", [0.00 0.00 0.00], ...
    "Margin", 6);
end

function styleAxisLabel(labelHandle, theme)
set(labelHandle, "Color", theme.textColor);
end
```

Question2.m

```
clear;
clc;
close all;

rng(5142, "twister");
params = getSystemParameters();
theme = getPlotTheme();
[~, preambleTd] = buildRepeatedPreamble(params);

numFrames = 2;
snrDb = 6;
timingLag = params.N / 2;
trueWindowStarts0 = params.cpLen + (0:numFrames-1) * params.frameSymbols * params.symbolLen;

txStream = buildFrameStream(params, preambleTd, numFrames);

[metricSingle, decisionSingle, detectedSingle0] = simulateTimingCase(txStream, 1, snrDb, params, timingLag, numFrames);
channelImpulseResponse = drawChannel(params);
[metricMultipath, decisionMultipath, detectedMultipath0] = simulateTimingCase(txStream, channelImpulseResponse, snrDb, params, t
```

```
figure("Name", "Question 2", "Color", theme.figureColor);
tiledlayout(2, 1, "TileSpacing", "compact", "Padding", "compact");

nexttile;
plotTimingMetric(metricSingle, decisionSingle, detectedSingle0, trueWindowStarts0, ...
    "Question 2(a): SC timing metric for a single-tap channel", theme);

nexttile;
plotTimingMetric(metricMultipath, decisionMultipath, detectedMultipath0, trueWindowStarts0, ...
    "Question 2(b): SC timing metric for the given multipath channel", theme);

fprintf("Question 2\n");
fprintf("Single-tap detected FFT window starts (0-based samples): %s\n", mat2str(detectedSingle0));
fprintf("Multipath detected FFT window starts (0-based samples): %s\n", mat2str(detectedMultipath0));
fprintf("In multipath, the plateau broadens because delayed paths keep the correlation high over a wider region.\n");
fprintf("A safe FFT window is still chosen inside the CP-supported high-metric region around each preamble.\n");

function params = getSystemParameters()
params.N = 512;
params.bandwidth = 5.12e6;
params.Ts = 1 / params.bandwidth;
params.cpLen = round(6.25e-6 / params.Ts);
params.frameSymbols = 5;
params.symbolLen = params.N + params.cpLen;
params.usefulSubcarriers = [-240:-1, 1:240];
params.usefulCarrierBins = mod(params.usefulSubcarriers, params.N) + 1;
params.pathDelays = [0 7 13 18 21 26];
pathPowersDb = [-3 0 -1 -4 -9 -17];
pathPowersLinear = 10.^(pathPowersDb / 10);
params.pathPowers = pathPowersLinear / sum(pathPowersLinear);
end

function [freqGrid, timeBlock, blockWithCp] = buildRepeatedPreamble(params)
activeSubcarriers = params.usefulSubcarriers(mod(params.usefulSubcarriers, 8) == 0);
freqGrid = zeros(params.N, 1);
freqGrid(mod(activeSubcarriers, params.N) + 1) = randomQpsk(numel(activeSubcarriers));
timeBlock = ifft(freqGrid) * sqrt(params.N);
blockWithCp = [timeBlock(end-params.cpLen+1:end); timeBlock];
end

function txStream = buildFrameStream(params, preambleTd, numFrames)
preambleWithCp = [preambleTd(end-params.cpLen+1:end); preambleTd];
txStream = [];

for frameIdx = 1:numFrames
    txStream = [txStream; preambleWithCp];

    for symbolIdx = 1:params.frameSymbols-1
        dataGrid = zeros(params.N, 1);
        dataGrid(params.usefulCarrierBins) = randomQpsk(numel(params.usefulCarrierBins));
        timeBlock = ifft(dataGrid) * sqrt(params.N);
        txStream = [txStream; timeBlock(end-params.cpLen+1:end); timeBlock];
    end
end
end

function [metric, decisionMetric, detectedWindows0] = simulateTimingCase(txStream, channelImpulseResponse, snrDb, params, timing
receivedClean = conv(txStream, channelImpulseResponse(:));
noiseVariance = 10^(-snrDb / 10);
noise = sqrt(noiseVariance / 2) * ...
    (randn(size(receivedClean)) + 1j * randn(size(receivedClean)));
received = receivedClean + noise;

metric = schmidlCoxMetric(received, timingLag);
decisionMetric = conv(metric, ones(params.cpLen, 1) / params.cpLen, "same");
detectedWindows0 = detectFrameWindows(decisionMetric, params, numFrames);
end
```

```
function metric = schmidlCoxMetric(received, timingLag)
numPositions = numel(received) - 2 * timingLag + 1;
metric = zeros(numPositions, 1);

for startIdx = 1:numPositions
    firstHalf = received(startIdx:startIdx+timingLag-1);
    secondHalf = received(startIdx+timingLag:startIdx+2*timingLag-1);
    pValue = sum(conj(firstHalf) .* secondHalf);
    rValue = sum(abs(secondHalf).^2);
    metric(startIdx) = abs(pValue)^2 / (rValue^2 + eps);
end
end

function detectedWindows0 = detectFrameWindows(decisionMetric, params, numFrames)
detectedWindows0 = zeros(1, numFrames);
centerToWindowOffset0 = floor((params.cpLen - 1) / 2);

for frameIdx = 1:numFrames
    frameStart0 = (frameIdx - 1) * params.frameSymbols * params.symbolLen;
    searchStart = frameStart0 + 1;
    searchStop = min(numel(decisionMetric), frameStart0 + params.symbolLen + params.cpLen + params.pathDelays(end));
    [~, localPeakIdx] = max(decisionMetric(searchStart:searchStop));
    detectedWindows0(frameIdx) = searchStart + localPeakIdx - 2 + centerToWindowOffset0;
end
end

function plotTimingMetric(metric, decisionMetric, detectedWindows0, trueWindowStarts0, plotTitle, theme)
sampleAxis0 = 0:numel(metric)-1;
metric = metric / max(metric + eps);
decisionMetric = decisionMetric / max(decisionMetric + eps);

ax = gca;
ax.ColorOrder = theme.palette;
plot(sampleAxis0, metric, "LineWidth", 1.0, "Color", theme.palette(1, :), ...
    "DisplayName", "SC timing metric");
hold on;
plot(sampleAxis0, decisionMetric, "LineWidth", 1.2, "Color", theme.palette(2, :), ...
    "DisplayName", "CP-smoothed decision metric");
grid on;
xlabelHandle = xlabel("Sample index");
ylabelHandle = ylabel("Normalized metric");
titleHandle = title(plotTitle);
applyDarkAxes(ax, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);

yl = ylim(ax);
for idx = 1:numel(trueWindowStarts0)
    if idx == 1
        line(ax, [trueWindowStarts0(idx) trueWindowStarts0(idx)], yl, ...
            "Color", theme.palette(3, :), "LineStyle", "--", "LineWidth", 1.0, ...
            "DisplayName", "True FFT window");
    else
        line(ax, [trueWindowStarts0(idx) trueWindowStarts0(idx)], yl, ...
            "Color", theme.palette(3, :), "LineStyle", "--", "LineWidth", 1.0, ...
            "HandleVisibility", "off");
    end
end

for idx = 1:numel(detectedWindows0)
    if idx == 1
        line(ax, [detectedWindows0(idx) detectedWindows0(idx)], yl, ...
            "Color", theme.palette(5, :), "LineStyle", ":", "LineWidth", 1.0, ...
            "DisplayName", "Derived FFT window");
    else
        line(ax, [detectedWindows0(idx) detectedWindows0(idx)], yl, ...
```

```
            "Color", theme.palette(5, :), "LineStyle", ":", "LineWidth", 1.0, ...
            "HandleVisibility", "off");
    end
end

legendHandle = legend("show", "Location", "best");
styleLegend(legendHandle, theme);
hold off;
end

function qpskSymbols = randomQpsk(symbolCount)
qpskSymbols = exp(1j * (pi / 4 + (pi / 2) * randi([0 3], symbolCount, 1)));
end

function channelImpulseResponse = drawChannel(params)
channelImpulseResponse = zeros(params.pathDelays(end) + 1, 1);
taps = sqrt(params.pathPowers(:) / 2) .* ...
    (randn(numel(params.pathDelays), 1) + 1j * randn(numel(params.pathDelays), 1));
channelImpulseResponse(params.pathDelays + 1) = taps;
end

function theme = getPlotTheme()
theme.figureColor = [0.00 0.00 0.00];
theme.axesColor = [0.05 0.05 0.05];
theme.textColor = [1.00 1.00 1.00];
theme.gridColor = [0.45 0.45 0.45];
theme.palette = [ ...
    0.15 0.85 1.00; ...
    1.00 0.72 0.20; ...
    0.35 1.00 0.45; ...
    1.00 0.40 0.75; ...
    1.00 0.32 0.32; ...
    0.60 0.65 1.00; ...
    0.20 1.00 0.85; ...
    1.00 0.92 0.30];
end

function applyDarkAxes(ax, theme)
set(ax, "Color", theme.axesColor, ...
    "XColor", theme.textColor, ...
    "YColor", theme.textColor, ...
    "GridColor", theme.gridColor, ...
    "MinorGridColor", theme.gridColor, ...
    "GridAlpha", 0.35, ...
    "MinorGridAlpha", 0.18, ...
    "LineWidth", 1.0);
box(ax, "on");
end

function styleTitle(titleHandle, theme)
set(titleHandle, "Color", theme.textColor, ...
    "BackgroundColor", [0.00 0.00 0.00], ...
    "EdgeColor", [0.00 0.00 0.00], ...
    "Margin", 6);
end

function styleAxisLabel(labelHandle, theme)
set(labelHandle, "Color", theme.textColor);
end

function styleLegend(legendHandle, theme)
set(legendHandle, "TextColor", theme.textColor, ...
    "Color", theme.axesColor, ...
    "EdgeColor", theme.gridColor);
end
```

Question3.m

```
clear;
clc;
close all;

rng(5143, "twister");
params = getSystemParameters();
theme = getPlotTheme();

snrDbList = 0:3:30;
numTrials = 100;
pilotSymbols = randomQpsk(params.numPilots);

pilotBins0 = mod(params.pilotSubcarriers, params.N);
usefulBins0 = mod(params.usefulSubcarriers, params.N);

aPilotCp = exp(-1j * 2 * pi / params.N * (pilotBins0(:) * (0:params.cpLen-1)));
aUsefulCp = exp(-1j * 2 * pi / params.N * (usefulBins0(:) * (0:params.cpLen-1)));
aPilotKnown = exp(-1j * 2 * pi / params.N * (pilotBins0(:) * params.pathDelays));
aUsefulKnown = exp(-1j * 2 * pi / params.N * (usefulBins0(:) * params.pathDelays));

msePilotZf = zeros(size(snrDbList));
mseLinear = zeros(size(snrDbList));
mseFft = zeros(size(snrDbList));
mseFftWindowed = zeros(size(snrDbList));
mseMls = zeros(size(snrDbList));
mseDelayAware = zeros(size(snrDbList));
representativeSNR = 12;
representativeCaptured = false;

for snrIdx = 1:numel(snrDbList)
    snrDb = snrDbList(snrIdx);
    noiseVariance = 10^(-snrDb / 10);
    errorAccumulator = zeros(1, 6);

    for trialIdx = 1:numTrials
        channelImpulseResponse = drawChannel(params);
        trueCfrFull = fft(channelImpulseResponse, params.N);
        trueCfrPilots = trueCfrFull(params.pilotCarrierBins);
        trueCfrUseful = trueCfrFull(params.usefulCarrierBins);

        noise = sqrt(noiseVariance / 2) * ...
            (randn(params.numPilots, 1) + 1j * randn(params.numPilots, 1));
        receivedPilots = pilotSymbols .* trueCfrPilots + noise;
        zfPilots = receivedPilots ./ pilotSymbols;

        linearEstimate = interpolatePilots(zfPilots, params);
        fftEstimate = dftDenoise(zfPilots, params, false);
        fftWindowedEstimate = dftDenoise(zfPilots, params, true);
        mlsTapEstimate = aPilotCp \ zfPilots;
        mlsEstimate = aUsefulCp * mlsTapEstimate;
        delayAwareTapEstimate = aPilotKnown \ zfPilots;
        delayAwareEstimate = aUsefulKnown * delayAwareTapEstimate;

        if ~representativeCaptured && snrDb == representativeSNR && trialIdx == 1
            representativeCaptured = true;
            representativeTrueCfr = trueCfrUseful;
            representativeZfPilots = zfPilots;
            representativeLinear = linearEstimate;
            representativeFftWindowed = fftWindowedEstimate;
            representativeMls = mlsEstimate;
            representativeDelayAware = delayAwareEstimate;
        end

        errorAccumulator(1) = errorAccumulator(1) + mean(abs(trueCfrPilots - zfPilots).^2);
        errorAccumulator(2) = errorAccumulator(2) + mean(abs(trueCfrUseful - linearEstimate).^2);
        errorAccumulator(3) = errorAccumulator(3) + mean(abs(trueCfrUseful - fftEstimate).^2);
```

```
        errorAccumulator(4) = errorAccumulator(4) + mean(abs(trueCfrUseful - fftWindowedEstimate).^2);
        errorAccumulator(5) = errorAccumulator(5) + mean(abs(trueCfrUseful - mlsEstimate).^2);
        errorAccumulator(6) = errorAccumulator(6) + mean(abs(trueCfrUseful - delayAwareEstimate).^2);
    end

    msePilotZf(snrIdx) = errorAccumulator(1) / numTrials;
    mseLinear(snrIdx) = errorAccumulator(2) / numTrials;
    mseFft(snrIdx) = errorAccumulator(3) / numTrials;
    mseFftWindowed(snrIdx) = errorAccumulator(4) / numTrials;
    mseMls(snrIdx) = errorAccumulator(5) / numTrials;
    mseDelayAware(snrIdx) = errorAccumulator(6) / numTrials;
end

figure("Name", "Question 3", "Color", theme.figureColor);
ax1 = axes();
plot(ax1, snrDbList, 10 * log10(msePilotZf + eps), "o-", "LineWidth", 1.1, ...
    "MarkerSize", 5, "Color", theme.palette(1, :), ...
    "MarkerFaceColor", theme.palette(1, :), "MarkerEdgeColor", theme.palette(1, :)); hold on;
plot(ax1, snrDbList, 10 * log10(mseLinear + eps), "s-", "LineWidth", 1.1, ...
    "MarkerSize", 5, "Color", theme.palette(2, :), ...
    "MarkerFaceColor", theme.palette(2, :), "MarkerEdgeColor", theme.palette(2, :));
plot(ax1, snrDbList, 10 * log10(mseFft + eps), "d-", "LineWidth", 1.1, ...
    "MarkerSize", 5, "Color", theme.palette(3, :), ...
    "MarkerFaceColor", theme.palette(3, :), "MarkerEdgeColor", theme.palette(3, :));
plot(ax1, snrDbList, 10 * log10(mseFftWindowed + eps), "^-", "LineWidth", 1.1, ...
    "MarkerSize", 5, "Color", theme.palette(4, :), ...
    "MarkerFaceColor", theme.palette(4, :), "MarkerEdgeColor", theme.palette(4, :));
plot(ax1, snrDbList, 10 * log10(mseMls + eps), "v-", "LineWidth", 1.1, ...
    "MarkerSize", 5, "Color", theme.palette(5, :), ...
    "MarkerFaceColor", theme.palette(5, :), "MarkerEdgeColor", theme.palette(5, :));
plot(ax1, snrDbList, 10 * log10(mseDelayAware + eps), "*-", "LineWidth", 1.1, ...
    "MarkerSize", 6, "Color", theme.palette(6, :), ...
    "MarkerFaceColor", theme.palette(6, :), "MarkerEdgeColor", theme.palette(6, :));
grid on;
xlabelHandle = xlabel("SNR (dB)");
ylabelHandle = ylabel("10 log_{10}(MSE)");
titleHandle = title("Question 3: OFDM channel-estimation performance");
applyDarkAxes(ax1, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);
legendHandle = legend({ ...
    "ZF on pilot tones only", ...
    "Linear interpolation", ...
    "FFT-based interpolation", ...
    "FFT-based interpolation with tapering", ...
    "Modified LS using tap support < N_{CP}", ...
    "Delay-aware LS with known tap locations"}, ...
    "Location", "southwest");
styleLegend(legendHandle, theme);
hold off;

figure("Name", "Question 3 Representative CFR", "Color", theme.figureColor);
ax2 = axes();
plot(ax2, params.usefulSubcarriers, 20 * log10(abs(representativeTrueCfr) + eps), "-", ...
    "LineWidth", 1.4, "Color", theme.palette(1, :)); hold on;
plot(params.pilotSubcarriers, 20 * log10(abs(representativeZfPilots) + eps), "o", ...
    "LineWidth", 1.0, "MarkerSize", 4, "Color", theme.palette(2, :), ...
    "MarkerFaceColor", theme.palette(2, :), "MarkerEdgeColor", theme.palette(2, :));
plot(params.usefulSubcarriers, 20 * log10(abs(representativeLinear) + eps), "--", ...
    "LineWidth", 1.0, "Color", theme.palette(3, :));
plot(params.usefulSubcarriers, 20 * log10(abs(representativeFftWindowed) + eps), "-.", ...
    "LineWidth", 1.1, "Color", theme.palette(4, :));
plot(params.usefulSubcarriers, 20 * log10(abs(representativeMls) + eps), ":", ...
    "LineWidth", 1.3, "Color", theme.palette(5, :));
plot(params.usefulSubcarriers, 20 * log10(abs(representativeDelayAware) + eps), "-", ...
    "LineWidth", 1.2, "Color", theme.palette(6, :));
grid on;
```

```
xlabelHandle = xlabel("Subcarrier label n");
ylabelHandle = ylabel("20 log_{10}|G[n]|");
titleHandle = title("Representative CFR estimate comparison at 12 dB");
applyDarkAxes(ax2, theme);
styleAxisLabel(xlabelHandle, theme);
styleAxisLabel(ylabelHandle, theme);
styleTitle(titleHandle, theme);
legendHandle = legend({ ...
    "True CFR", ...
    "ZF at pilot tones", ...
    "Linear interpolation", ...
    "FFT interpolation with tapering", ...
    "Modified LS", ...
    "Delay-aware LS"}, ...
    "Location", "best");
styleLegend(legendHandle, theme);
hold off;

fprintf("Question 3\n");
fprintf("Pilots are placed every 8 subcarriers at n = +/-8, +/-16, ..., +/-240.\n");
fprintf("The plotted MSE values are averaged per evaluated carrier and then averaged across J = 100 channel realizations.\n");
fprintf("Expected ordering at moderate-to-high SNR: delay-aware LS <= mLS <= FFT-based <= linear interpolation.\n");

function params = getSystemParameters()
params.N = 512;
params.bandwidth = 5.12e6;
params.Ts = 1 / params.bandwidth;
params.cpLen = round(6.25e-6 / params.Ts);
params.usefulSubcarriers = [-240:-1, 1:240];
params.usefulCarrierBins = mod(params.usefulSubcarriers, params.N) + 1;
params.pilotSubcarriers = [-240:8:-8, 8:8:240];
params.pilotCarrierBins = mod(params.pilotSubcarriers, params.N) + 1;
params.numPilots = numel(params.pilotSubcarriers);
params.pathDelays = [0 7 13 18 21 26];
pathPowersDb = [-3 0 -1 -4 -9 -17];
pathPowersLinear = 10.^(pathPowersDb / 10);
params.pathPowers = pathPowersLinear / sum(pathPowersLinear);
end

function linearEstimate = interpolatePilots(zfPilots, params)
realPart = interp1(params.pilotSubcarriers, real(zfPilots).', params.usefulSubcarriers, "linear");
imagPart = interp1(params.pilotSubcarriers, imag(zfPilots).', params.usefulSubcarriers, "linear");
linearEstimate = realPart(:) + 1j * imagPart(:);
end

function cfrEstimate = dftDenoise(zfPilots, params, useWindow)
pilotGrid = zeros(params.N, 1);
pilotGrid(params.pilotCarrierBins) = zfPilots;
impulseResponse = ifft(pilotGrid);
scaleFactor = 8;
filteredImpulseResponse = zeros(params.N, 1);
filteredImpulseResponse(1:params.cpLen) = impulseResponse(1:params.cpLen) * scaleFactor;

if useWindow
    taperLength = min(8, params.cpLen);
    window = ones(params.cpLen, 1);
    taperIndex = (0:taperLength-1).';
    window(end-taperLength+1:end) = 0.5 * (1 + cos(pi * (taperIndex + 1) / taperLength));
    filteredImpulseResponse(1:params.cpLen) = filteredImpulseResponse(1:params.cpLen) .* window;
end

fullCfrEstimate = fft(filteredImpulseResponse);
cfrEstimate = fullCfrEstimate(params.usefulCarrierBins);
end

function qpskSymbols = randomQpsk(symbolCount)
qpskSymbols = exp(1j * (pi / 4 + (pi / 2) * randi([0 3], symbolCount, 1)));
end
```

```
function channelImpulseResponse = drawChannel(params)
channelImpulseResponse = zeros(params.pathDelays(end) + 1, 1);
taps = sqrt(params.pathPowers(:) / 2) .* ...
    (randn(numel(params.pathDelays), 1) + 1j * randn(numel(params.pathDelays), 1));
channelImpulseResponse(params.pathDelays + 1) = taps;
end

function theme = getPlotTheme()
theme.figureColor = [0.00 0.00 0.00];
theme.axesColor = [0.05 0.05 0.05];
theme.textColor = [1.00 1.00 1.00];
theme.gridColor = [0.45 0.45 0.45];
theme.palette = [ ...
    0.15 0.85 1.00; ...
    1.00 0.72 0.20; ...
    0.35 1.00 0.45; ...
    1.00 0.40 0.75; ...
    1.00 0.32 0.32; ...
    0.60 0.65 1.00; ...
    0.20 1.00 0.85; ...
    1.00 0.92 0.30];
end

function applyDarkAxes(ax, theme)
set(ax, "Color", theme.axesColor, ...
    "XColor", theme.textColor, ...
    "YColor", theme.textColor, ...
    "GridColor", theme.gridColor, ...
    "MinorGridColor", theme.gridColor, ...
    "GridAlpha", 0.35, ...
    "MinorGridAlpha", 0.18, ...
    "LineWidth", 1.0);
box(ax, "on");
end

function styleTitle(titleHandle, theme)
set(titleHandle, "Color", theme.textColor, ...
    "BackgroundColor", [0.00 0.00 0.00], ...
    "EdgeColor", [0.00 0.00 0.00], ...
    "Margin", 6);
end

function styleAxisLabel(labelHandle, theme)
set(labelHandle, "Color", theme.textColor);
end

function styleLegend(legendHandle, theme)
set(legendHandle, "TextColor", theme.textColor, ...
    "Color", theme.axesColor, ...
    "EdgeColor", theme.gridColor);
end
```